

Dynamic Programming

@author Yangxin Wu

Last edited: 2025.06.12

- [Intro](#)
- [Matrix Multiplication](#)
- [Longest Common Subsequence](#)
- [0-1 Knapsack](#)
- [Optimal Binary Search Tree](#)

△ 个人觉得 DP 知识性比较弱，实践性比较强，不刷个上百道算法题都不好意思说会写 DP。考试应该不会让手搓程序（? ），大概有个感觉应该就行。

Intro

动态规划是一种通过把原问题分解为相对简单的子问题的方式求解复杂问题的方法。由于动态规划并不是某种具体的算法，而是一种解决特定问题的方法，因此它会出现在各式各样的数据结构中，实际情况也因而更为繁杂。

特点：

1. 把原始问题划分成一系列子问题
2. 求解每个子问题仅一次，并将其结果保存在一个表中，以后用到时直接存取，不重复计算，节省计算时间
3. 自底向上地计算

适用范围：可分解为多个子问题且子问题的解被重复使用的优化问题。

使用条件：

1. 最优子结构
 - 当一个问题的最优解包含了子问题的最优解时，我们称这个问题具有最优子结构：原问题对应于子问题的解是该子问题的最优解；
 - 最优子结构使得我们能自底向上地完成求解过程。
2. 重叠子问题
 - 在问题的求解过程中，很多子问题的解将被多次使用。通过自底向上求解，避免重复计算。

设计步骤：

1. 分析最优解的结构，是否具有“最优子结构”和“重叠子问题”；
2. 递归地定义最优值的代价（注意递归方程初始条件）；

3. 自底向上地计算最优解的代价保存之，并获取构造最优值的信息；

4. 根据构造最优值的信息构造最优解。

Matrix Multiplication

问题定义：

输入： $\langle A_1, A_2, \dots, A_n \rangle$, $A_i \in \mathbb{R}^{p_{i-1} \times p_i}$

输出：计算 $A_1 \times A_2 \times \dots \times A_n$ 的最小代价方法

若 A 是 $p \times q$ 矩阵， B 是 $q \times r$ 矩阵， $A \times B$ 的代价如何计算？

回想一下 CSAPP 中对矩阵乘法的优化，我们通常需要三层 for 循环来实现矩阵乘法。由于循环次数分别为 p, q, r ，我们自然可以得到代价是 $O(pqr)$ 。

那么对于一个矩阵链的连乘，我们自然也可以这样按照 $A_1 \times A_2 \times \dots \times A_n$ 的顺序依次计算代价。但是其实我们不难意识到，计算顺序实际上会影响代价。例如：对于矩阵 $A_1 \in \mathbb{R}^{10 \times 100}$, $A_2 \in \mathbb{R}^{100 \times 5}$, $A_3 \in \mathbb{R}^{5 \times 50}$ ，我们有

$$\begin{aligned} T((A_1 \times A_2) \times A_3) &= 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500 \\ T(A_1 \times (A_2 \times A_3)) &= 100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000 \end{aligned}$$

因此为了求出矩阵乘法的最小代价，我们需要找到一种特定的计算顺序。

记 $A_{i \sim j} = A_i \times A_{i+1} \times \dots \times A_j$, $\text{cost}(A_{i \sim j})$ 为计算 $A_{i \sim j}$ 的代价。

下面我们给出本问题的最优子结构。

假设 $A_i A_{i+1} \dots A_j$ 的最优括号化方案的分割点在 A_k 和 A_{k+1} 之间，当我们继续对子问题 $A_i A_{i+1} \dots A_k$ 进行括号化时，我们应该直接采用独立求解它时所得的最优方案。

证明：采用反证法。假设存在一个比独立求解 $A_i A_{i+1} \dots A_k$ 所得括号化方案更优的方案 S' ，那么替换后得到的总代价将是：(次优方案 S' 的代价) + $\text{cost}(A_{k+1 \sim j})$ + 最后一步相乘的代价。由于总代价比替换前更低， S' 的代价比原方案 S 的代价更低，这与 S 是单独求解 $A_i A_{i+1} \dots A_k$ 的最优括号化方案不符。因此为了保持整体问题的最优性，其子问题 $A_i A_{i+1} \dots A_k$ 必须也采用其独立求解时所得的最优方案。

记 $m[i, j] = \text{cost}(A_{i \sim j})$ ，那么 $m[1, n] = \text{cost}(A_{1 \sim n})$ 。

假设 $A_1 \dots A_k A_{k+1} \dots A_n$ 是最优解 ($k \in [i, j]$)，那么代价的递推方程可以写为：

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{if } i < j \end{cases}$$

复杂度分析

时间复杂度： $O(n^3)$

- 计算代价的时间：(i, j, k) 三层循环，每层至多 $n-1$ 步，复杂度为 $O(n^3)$
- 构造最优解的时间： $O(n)$
空间复杂度：使用数组 p, m, S ，需要空间 $O(n^2)$

示例代码如下：

```

/**
 * @brief 计算矩阵链乘法的最小代价
 * @param p 存储矩阵维度的数组。
 * p[i-1] 是 A_i 的行数，p[i] 是 A_i 的列数。
 * p 的长度为 n+1，表示有 n 个矩阵。
 * @param n 矩阵链中矩阵的数量。
 * @return 最小乘法代价。
 */
int matrixChainOrder(int p[], int n) {
    /*
     * m[i][j] 存储计算矩阵 A_i...A_j 所需的最小乘法次数。
     * s[i][j] 存储 A_i...A_j 最佳括号化方案的分割点 k。
     * 数组大小 n+1 是因为矩阵编号从 1 到 n，对应 p 数组的索引。
     * 所以 m 和 s 数组的索引范围是 1 到 n。
     */
    int m[n+1][n+1];
    int s[n+1][n+1];

    int i, j, k, len, q;

    // 单个矩阵的乘法代价为 0
    for (i = 1; i <= n; i++) {
        m[i][i] = 0;
    }

    // len 是当前考虑的矩阵链的长度
    for (len = 2; len <= n; len++) {
        // i 是链的起始矩阵索引
        // j 是链的结束矩阵索引
        for (i = 1; i <= n - len + 1; i++) {
            j = i + len - 1;
            m[i][j] = INT_MAX;

            // k 是分割点，从起始索引到结束索引遍历
            for (k = i; k <= j-1; k++) {
                // 计算当前分割点 k 的代价
                q = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j];

                // 如果当前代价 q 更小，则更新 m[i][j] 和 s[i][j]
                if (q < m[i][j]) {
                    m[i][j] = q;
                    // 记录最优分割点
                    s[i][j] = k;
                }
            }
        }
    }

    // 返回 A_1...A_n 的最小乘法代价

```

```

    return m[1][n];
}

```

Longest Common Subsequence

问题定义：

输入： $X = (x_1, x_2, \dots, x_n)$, $Y = (y_1, y_2, \dots, y_m)$

输出： $Z = X$ 与 Y 的最长公共子序列

定义序列 X 的第 i 前缀 X_i 是一个序列，为 $X_i = (x_1, x_2, \dots, x_i)$ 。

下面分析本问题的最优子结构。

设 $X = (x_1, \dots, x_m)$, $Y = (y_1, \dots, y_n)$ 是两个序列，若 $Z = (z_1, \dots, z_k)$ 是 X 与 Y 的 LCS ，那么我们有：

1. 如果 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ， Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的 LCS ，即

$$LCS_{XY} = LCS_{X_{m-1}Y_{n-1}} + \langle x_m = y_n \rangle$$

2. 如果 $x_m \neq y_n$ ，且 $z_k \neq x_m$ ，则 Z 是 X_{m-1} 和 Y 的 LCS ，即 $LCS_{XY} = LCS_{X_{m-1}Y}$ ；
3. 如果 $x_m \neq y_n$ ，且 $z_k \neq y_n$ ，则 Z 是 X 与 Y_{n-1} 的 LCS ，即 $LCS_{XY} = LCS_{XY_{n-1}}$ 。

而事实上，我们并不知道 z_k 是什么，所以当 $x_m \neq y_n$ 时：

$$LCS_{XY} = \max(LCS_{X_{m-1}Y}, LCS_{XY_{n-1}})。$$

故 X 和 Y 的 LCS 的最优解结构为

$$LCS_{XY} = \begin{cases} LCS_{X_{m-1}Y_{n-1}} + \langle x_m = y_n \rangle & \text{if } x_m = y_n \\ \max(LCS_{X_{m-1}Y}, LCS_{XY_{n-1}}) & \text{if } x_m \neq y_n \end{cases}$$

定义： $C_{i,j}$ = 前缀 X_i 与前缀 Y_j 的 LCS 的长度。那么 $C_{i,j}$ 的递归方程为

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ 或 } j = 0 \\ C[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ 且 } x_i = y_j \\ \max(C[i, j-1], C[i-1, j]) & \text{if } i, j > 0 \text{ 且 } x_i \neq y_j \end{cases}$$

示例代码如下：

```

// 定义用于回溯的常量
#define DIAG 0 // 对角线移动 (x_i == y_j)
#define UP 1 // 向上移动 (x_i != y_j, 忽略 x_i)
#define LEFT 2 // 向左移动 (x_i != y_j, 忽略 y_j)

/**
 * @brief 计算两个字符串的最长公共子序列的长度并填充方向表
 * @param X 第一个字符串
 * @param Y 第二个字符串
 * @param c 存储LCS长度的动态规划表 (m+1)x(n+1)
 * @param b 存储回溯方向的表 (m+1)x(n+1)
 * @return LCS的长度
 */

```

```

int LCS_Length(char* X, char* Y, int** c, int** b) {
    int m = strlen(X); // 序列 X 的长度
    int n = strlen(Y); // 序列 Y 的长度

    // c[i][j] 存储 X_i 和 Y_j 的 LCS 长度
    c = (int**)malloc((m + 1) * sizeof(int*));
    for (int i = 0; i <= m; i++) {
        c[i] = (int*)malloc((n + 1) * sizeof(int));
    }

    // b[i][j] 存储从 c[i][j] 回溯到上一个状态的方向
    b = (int**)malloc((m + 1) * sizeof(int*));
    for (int i = 0; i <= m; i++) {
        b[i] = (int*)malloc((n + 1) * sizeof(int));
    }

    // 当其中一个序列为空时, LCS 长度为 0
    for (int i = 0; i <= m; i++) {
        c[i][0] = 0;
    }
    for (int j = 0; j <= n; j++) {
        c[0][j] = 0;
    }

    // 填充 c 和 b 表
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i - 1] == Y[j - 1]) {
                c[i][j] = c[i - 1][j - 1] + 1;
                // 对角线移动, 表示匹配
                b[i][j] = DIAG;
            } else if (c[i - 1][j] >= c[i][j - 1]) {
                c[i][j] = c[i - 1][j];
                // 向上移动, 表示忽略 X_i
                b[i][j] = UP;
            } else {
                c[i][j] = c[i][j - 1];
                // 向左移动, 表示忽略 Y_j
                b[i][j] = LEFT;
            }
        }
    }

    // 最终的LCS长度
    int lcs_len = c[m][n];

    return lcs_len;
}

```

0-1 Knapsack

起源：「USACO07 DEC」 Charm Bracelet

有 n 个物品和一个容量为 W 的背包，每个物品有重量 w_i 和价值 v_i 两种属性，要求选若干物品放入背包使背包中物品的总价值最大，且背包中物品的总重量不超过背包的容量。

记 $f_{i,j}$ 为在只能放前 i 个物品的情况下，容量为 j 的背包所能达到的最大总价值。

假设当前已经处理好了前 $i-1$ 个物品的所有状态，那么对于第 i 个物品：

1. 如果决定不放入背包，那么背包的剩余容量和背包中物品的总价值不会改变。此时的最大价值就等于在只考虑前 $i-1$ 个物品、且背包容量仍为 j 时的最大价值，即 $f_{i-1,j}$ ；
2. 如果决定放入背包，首先要检查背包是否有足够的容量来装它，即 j 是否大于等于 w_i 。
 1. 如果容量足够，那么放入后背包的剩余容量会减小 w_i ，背包中物品的总价值会增大 v_i 。此时的最大价值应该是在只考虑前 $i-1$ 个物品、且背包容量为 $j-w_i$ 时的最大价值，再加上第 i 个物品的价值 v_i ，即 $f_{i-1,j-w_i} + v_i$ 。由此可以得出状态转移方程：

$$f_{i,j} = \max(f_{i-1,j}, f_{i-1,j-w_i} + v_i)$$

我们不难发现，对于 f_j 有影响的只有 f_{i-1} ，也就是对于原先存储状态的二维矩阵，只有前一行才会对后一行产生影响。因此我们可以去掉第一维，直接用 f_j 来表示处理到当前物品时背包容量为 j 的最大价值，得出以下方程：

$$f_j = \max(f_j, f_{j-w_i} + v_i)$$

这样可以显著的降低空间复杂度。

```
/**
 * @brief 使用一维DP数组解决0/1背包问题（空间优化）
 * @param weights 物品重量数组 (w_1, w_2, ..., w_n)
 * @param values 物品价值数组 (v_1, v_2, ..., v_n)
 * @param n 物品数量
 * @param W 背包容量
 * @return 能放入背包的最大总价值
 */
int knapsack_01(int weights[], int values[], int n, int W) {
    // f[j] 表示当前考虑物品情况下，容量为 j 的背包所能达到的最大价值
    int* f = (int*)malloc((W + 1) * sizeof(int));

    // 初始化 f 数组所有元素为 0，表示当背包容量为 j 且不放入任何物品时价值为 0
    for (int j = 0; j <= W; j++) {
        f[j] = 0;
    }

    // 遍历每个物品
    for (int i = 1; i <= n; i++) {
        // 当前物品的重量和价值
        int current_weight = weights[i-1];
```

```

    int current_value = values[i-1];

    // 倒序遍历背包容量 j
    // j 必须大于等于 current_weight 才能考虑放入当前物品
    for (int j = W; j >= current_weight; j--) {
        f[j] = MAX(f[j], f[j - current_weight] + current_value);
    }
}

int max_value = f[W];

free(f);

return max_value;
}

```

你可能会疑惑，这里为什么要用倒序遍历剩余容量 j 。
 让我们以一个具体例子来模拟：

- 物品 1: $w=2, v=3$
- 物品 2: $w=3, v=4$
- 背包容量 $W=5$

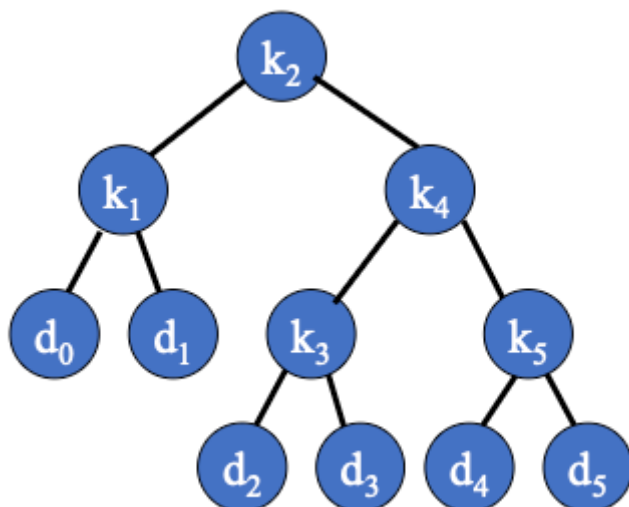
处理物品 1 ($w=2, v=3$) 时，假设我们使用正序遍历 j 从 2 遍历到 5：

- $j=2$: $f[2] = \text{MAX}(f[2], f[2-2] + 3) = \text{MAX}(f[2], f[0] + 3) = \text{MAX}(0, 0+3) = 3$
- $j=3$: $f[3] = \text{MAX}(f[3], f[3-2] + 3) = \text{MAX}(f[3], f[1] + 3) = \text{MAX}(0, 0+3) = 3$
- $j=4$: $f[4] = \text{MAX}(f[4], f[4-2] + 3) = \text{MAX}(f[4], f[2] + 3) = \text{MAX}(0, 3+3) = 6$

问题出现！这里在计算 $f[4]$ 时，用到的 $f[2]$ 在 $j=2$ 时已经被更新过了。也就是说我们多次放入了物品 1，这与问题的定义不符。

因此倒序遍历容量确保了每个物品最多只被选择一次。

Optimal Binary Search Tree



问题定义

输入：

$K = \{k_1, k_2, \dots, k_n\}$ ，已排序的关键字序列，且 $k_1 < k_2 < \dots < k_n$

$D = \{d_0, d_1, \dots, d_n\}$ ，伪关键字序列，且根据 BST 的性质有

- d_0 对应区间 $(-\infty, k_1)$
- d_i 对应区间 (k_i, k_{i+1})
- d_n 对应区间 (k_n, ∞)

$P = \{p_1, p_2, \dots, p_n\}$ ， p_i 为搜索关键字 k_i 的概率

$Q = \{q_0, q_1, \dots, q_n\}$ ， q_i 为搜索伪关键字 d_i 的概率

输出：

K 的最优二叉搜索树 T ，期望代价 $E(T)$ 最小。其中

$$E(T) = \sum_{i=1}^n (\text{depth}_T(k_i) + 1)p_i + \sum_{j=0}^n (\text{depth}_T(d_j) + 1)q_j$$

下分析本问题的最优子结构

如果最优二叉搜索树 T 具有包含关键字集合 $k_i, k_{i+1}, \dots, k_j$ 子树 T' ，则 T' 必是关于关键字集合 $k_i, k_{i+1}, \dots, k_j$ 子问题的最优解。证明与矩阵链乘法类似，这里不做赘述。

记：

$E(i, j)$ 为 $\{k_i, \dots, k_j\}$ 和 $\{d_{i-1}, d_i, \dots, d_j\}$ 的最优解 T_{ij} （包含这些元素的子树）的期望搜索代价。

$W(i, j)$ 为搜索 $\{k_i, \dots, k_j\}$ 和 $\{d_{i-1}, \dots, d_j\}$ 的概率之和。

分析可知：

1. 当 $j = i - 1$ 时，可知 T_{ij} 中只有叶结点 d_{i-1} ，则 $E(i, i - 1) = q_{i-1} = q_j$ 。
2. 当 $j \geq i$ 时，我们选择 $K_r \in \{k_i, \dots, k_j\}$ 作为该最优解的根节点。 K_r 将 $\{k_i, \dots, k_j\}$ 分解为 $\{k_i, \dots, k_{r-1}\}$ 和 $\{k_{r+1}, \dots, k_j\}$ 两个子问题，两个子问题可以分别递归求解，即我们有

$$\begin{aligned} E(i, j) &= P_r + (E(i, r - 1) + W(i, r - 1)) + (E(r + 1, j) + W(r + 1, j)) \\ &= E(i, r - 1) + E(r + 1, j) + (W(i, r - 1) + P_r + W(r + 1, j)) \\ &= E(i, r - 1) + E(r + 1, j) + W(i, j) \end{aligned}$$

由上可知，搜索代价递归方程为：

$$E(i, j) = \begin{cases} q_{i-1} = q_j & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{E(i, r - 1) + E(r + 1, j) + W(i, j)\} & \text{if } j \geq i \end{cases}$$

示例代码如下：

```
/**
 * @brief 计算最优二叉搜索树的最小期望搜索代价
 * @param p 关键字概率数组 (p[0]对应k1, p[1]对应k2, ..., p[n-1]对应kn)
 * @param q 伪关键字概率数组 (q[0]对应d0, q[1]对应d1, ..., q[n]对应dn)
 * @param n 关键字数量
 * @return 最小期望搜索代价
```



```

*/
double optimalBST(double p[], double q[], int n) {
    // 数组索引: i 从 1 到 n+1, j 从 0 到 n

    // E[i][j] 存储关键字 {k_i, ..., k_j} 和伪关键字 {d_{i-1}, ..., d_j} 的最
    优期望代价
    double** E = (double**)malloc((n + 2) * sizeof(double*));
    // W[i][j] 存储子树 {k_i, ..., k_j} 和 {d_{i-1}, ..., d_j} 的概率和
    double** W = (double**)malloc((n + 2) * sizeof(double*));
    // root[i][j] 存储子树 {k_i, ..., k_j} 的根节点索引 r
    int** root = (int**)malloc((n + 2) * sizeof(int*));

    for (int i = 0; i <= n + 1; i++) {
        E[i] = (double*)malloc((n + 1) * sizeof(double));
        W[i] = (double*)malloc((n + 1) * sizeof(double));
        root[i] = (int*)malloc((n + 1) * sizeof(int));
    }

    // 对于链长 len = 0 的情况 (j = i-1), 只有伪关键字
    for (int i = 1; i <= n + 1; i++) {
        E[i][i-1] = q[i-1];
        W[i][i-1] = q[i-1];
    }

    // len 是当前考虑的关键字链的长度 (从 1 到 n)
    for (int len = 1; len <= n; len++) {
        // i 是链的起始关键字索引 (从 1 到 n - len + 1)
        for (int i = 1; i <= n - len + 1; i++) {
            // j 是链的结束关键字索引
            int j = i + len - 1;

            W[i][j] = W[i][j-1] + p[j-1] + q[j];

            E[i][j] = DBL_MAX;
            // 遍历所有可能的根节点 r (从 i 到 j)
            for (int r = i; r <= j; r++) {
                double current_cost = E[i][r - 1] + E[r + 1][j] + W[i][j];

                // 如果当前代价更小, 则更新 E[i][j] 和 root[i][j]
                if (current_cost < E[i][j]) {
                    E[i][j] = current_cost;
                    // 记录最优根节点
                    root[i][j] = r;
                }
            }
        }
    }

    // 最终结果在 E[1][n]
    double min_expected_cost = E[1][n];
}

```

```
// 释放 W 表内存
for (int i = 0; i <= n + 1; i++) {
    free(W[i]);
}
free(W);

return min_expected_cost;
}
```